

Section 89 — Polynomial Evaluation And Subset Transforms

Change representation until the operation is pointwise

This lesson derives Problems A–C. Problem D is editorial-only.

A. Multipoint evaluation

For one point, Horner is linear; repeating it is quadratic. Build a product tree whose leaf i is $x - X_i$ and whose internal polynomial is the product of its children. At a node, replace P by its remainder modulo the node polynomial. Both children need only the remainder modulo their own product. At a leaf the remainder is the scalar $P(X_i)$.

Fast polynomial division reverses dividend and divisor, computes a truncated series inverse, recovers the quotient, then subtracts. Across one tree level the total polynomial size is linear, so the complete algorithm costs $O(M(n) \log n)$.

B. Interpolation is the transposed product tree

Let $M(x)$ be the product of all $(x - X_i)$. The Lagrange weight is

$$w_i = Y_i / M'(X_i).$$

Distinct points ensure the denominator is nonzero. Put w_i at leaves. At each internal node combine

$$\text{left_value} * \text{right_product} + \text{right_value} * \text{left_product}.$$

Induction says this is the sum of every leaf weight times all factors except its own. At the root it is exactly the interpolating polynomial.

C. XOR convolution

The Walsh–Hadamard transform uses butterflies $(u, v) \rightarrow (u+v, u-v)$. A bit independently records whether two input bits agree or differ, so XOR convolution becomes pointwise multiplication in transformed coordinates. Applying the same butterflies again yields n times the input; multiply by $\text{inverse}(n)$ to invert.

A unifying question

Convolution is difficult in the original basis because outputs mix many input pairs. Search for a representation where the operation is coordinatewise: values at roots for ordinary multiplication, Walsh characters for XOR, and subset-zeta ranks for disjoint union.

A small product-tree trace

For points X_0, X_1, X_2 , the leaves store $x-X_0$, $x-X_1$, and $x-X_2$ (plus an identity leaf if the implementation pads to a power of two). Their parents store products, and the root stores M . Starting with $P \bmod M$, send

```
current remainder mod left product
current remainder mod right product
```

to the children. Congruence is transitive because every child product divides its parent product. This is the whole correctness invariant; the fast division routine changes the running time, not the mathematics.

For interpolation, the leaf contribution is w_i . When merging two subtrees, multiply the left contribution by every factor belonging to the right subtree, and symmetrically for the right. Thus every leaf eventually acquires all factors except its own.

A two-bit Walsh trace

For four values, first butterfly pairs differing in bit 0, then pairs differing in bit 1. Each output coordinate is a signed sum determined by a bit character. Because the sign for $a \text{ xor } b$ is the product of the signs for a and b , XOR convolution becomes pointwise multiplication. Running the same stages again returns four times the input.

Do not interchange transforms:

- XOR uses sum/difference butterflies;
- OR uses sums over submasks;
- AND uses sums over supermasks;
- disjoint-union convolution needs both subset zeta transforms and popcount ranks, because ordinary zeta multiplication also counts overlaps.

Implementation checks

Product trees must preserve the original point order at leaves. Interpolation must reject or exclude repeated points, whereas evaluation permits them. Transform arrays must have length exactly a power of two covering every mask. For stress tests, compare against Horner evaluation, quadratic Lagrange interpolation, and the direct double loop for every mask operation at small dimensions.

Exercises

A–C cover product trees, interpolation, and XOR transforms. D introduces ranked subset convolution without advance exposure.